

Análise e Desenvolvimento de Sistemas

Gerenciamento Ágil de Projetos

Extreme Programming

Período: **08/05/2026** a **03/07/2026**

Em grupo | Todos os grupos abordam os 5 temas | 8 sprints + entrega final

Valores e Práticas do Extreme Programming (XP)

O **Extreme Programming (XP)** é uma metodologia ágil focada em melhorar a qualidade do software e facilitar adaptações rápidas às mudanças dos requisitos. O XP possui valores fundamentais e práticas que incentivam colaboração, simplicidade e entregas contínuas.

Valores do XP

Comunicação

A comunicação constante entre desenvolvedores e clientes ajuda a evitar erros e garante que todos entendam os objetivos do projeto.

Simplicidade

O desenvolvimento deve focar apenas no que é necessário no momento, evitando complexidade desnecessária.

Feedback

O feedback rápido permite corrigir problemas e melhorar funcionalidades continuamente.

Coragem

A equipe deve ter coragem para modificar, refatorar e melhorar o código sempre que necessário.

Respeito

Todos os membros da equipe devem respeitar opiniões e contribuições uns dos outros.

Práticas do XP

TDD (Test-Driven Development)

No TDD, os testes são criados antes da implementação do código.

Pair Programming

Dois desenvolvedores trabalham juntos no mesmo computador, um programando e outro revisando.

Integração Contínua

O código é integrado frequentemente ao projeto principal, com testes automáticos sendo executados constantemente.

Refatoração

Consiste em melhorar a estrutura do código sem alterar seu funcionamento.

Pequenas Releases

O sistema é entregue em pequenas versões frequentes para receber feedback rapidamente.

Exemplo de Fluxo no XP

1. O cliente solicita uma funcionalidade de login.
 2. A equipe cria primeiro os testes (TDD).
 3. Dois programadores trabalham juntos implementando a funcionalidade (Pair Programming).
 4. O código é enviado para o repositório.
 5. A Integração Contínua executa os testes automaticamente.
 6. Após aprovação, a funcionalidade é liberada para o cliente testar.
-

Exemplo simples de código TDD em Java

Classe de Teste

```
@Test
public void deveValidarLogin() {
    Login login = new Login();
    assertTrue(login.validar("admin", "123"));
}
```

Classe Login

```
public class Login {

    public boolean validar(String usuario, String senha) {
        return usuario.equals("admin") &&
            senha.equals("123");
    }
}

public class Login {
    public boolean validar(String usuario, String senha) {
        return usuario.equals("admin") && senha.equals("123");
    }
}
```

eXtreme Programming (XP) Estudo de caso: aplicação de ao menos 3 práticas XP em um projeto fictício de desenvolvimento de software. (PDF)

1. Test-Driven Development (TDD)

A equipe precisava desenvolver o módulo principal de "Criação de Tarefas" da plataforma. Seguindo o XP, os testes ditaram o design da funcionalidade.

Aplicação Prática: Antes de criar a rota no back-end (Node.js) ou a tabela no MySQL, o desenvolvedor escreveu um teste automatizado chamado `deveRejeitarTarefaSemTitulo()`. O teste inicialmente falhou (fase vermelha). O desenvolvedor então escreveu apenas o código de validação estritamente necessário para o teste passar (fase verde). Após garantir a aprovação, o código foi refatorado para manter a simplicidade e clareza, sem quebrar a funcionalidade.

2. Pair Programming (Programação em Par)

O desenvolvimento do "Quadro Kanban" interativo no front-end exigia uma lógica complexa de manipulação de estados no React. Em vez de isolar um desenvolvedor na tarefa, a equipe adotou a programação em par.

Aplicação Prática: Dois programadores compartilharam o mesmo computador. O primeiro atuou como "Piloto", digitando o código e focando na implementação imediata da interface e do arrastar-e-soltar (drag-and-drop) dos cartões. O segundo atuou como "Navegador", revisando a sintaxe do TypeScript em tempo real, sugerindo melhorias na arquitetura estrutural e antecipando possíveis problemas. Isso evitou refatorações futuras e acelerou a entrega da tela.

3. Integração Contínua

Com vários desenvolvedores atuando ao mesmo tempo nos módulos de login, cadastro e gestão de usuários, era fundamental garantir que os novos códigos não quebrassem as funcionalidades antigas do Scrunic.

Aplicação Prática: A equipe configurou uma ferramenta de automação no repositório do projeto. Diversas vezes ao dia, sempre que um desenvolvedor finalizava uma tarefa e enviava seu código (commit), o sistema de Integração Contínua era acionado. Ele compilava a aplicação e executava toda a suíte de testes do TDD automaticamente. Se alguma alteração gerasse um conflito com o banco MySQL ou fizesse algum teste falhar, o envio era rejeitado imediatamente, permitindo que a equipe corrigisse o erro na hora, em vez de acumular problemas para o final da semana.